
SIMVIS

Release 1.0.0

SC,AE,SB,BB

May 20, 2026

Contents:

1	simvis module	1
1.1	All sky simulations of Gaussian random sky signal and corresponding Visibilities	1
1.2	To run:	2
1.3	Overview	6
1.4	Module Functions	7

SIMVIS MODULE

1.1 All sky simulations of Gaussian random sky signal and corresponding Visibilities

For a given Angular Power Spectrum [APS, in mK^2 unit] of brightness temperature fluctuations or Power Spectrum [PS, in $\text{mK}^2 \text{ Mpc}^3$ unit] and an input UVFITS file, this outputs a new UVFITS file with simulated visibilities.

Our aim is to simulate the visibilities for a given telescope and for either a given Angular Power Spectrum or a given Power Spectrum.

For given Angular Power Spectrum C_ℓ , we generate multiple realizations of the sky signal at the centered frequency ν_c , and for a given Power Spectrum $P(k)$, we generate 3D sky signal for all frequency channels (frequency channel information taken from input UVFITS file). Afterwards by this simulated sky signal, we simulate the corresponding visibilities.

Visibilities that are simulated are in Jy(Jansky) unit. The visibility at a baseline $\vec{U}$ is given by:

$$\mathcal{V}(\vec{U}, \nu) = Q_\nu \int_{\text{UH}} d\Omega_{\hat{n}} T(\hat{n}, \nu) A(\Delta\hat{n}, \nu) \exp \left[2\pi i \vec{U} \cdot \Delta\hat{n} \right]$$

- $T(\hat{n}, \nu)$ is the surface brightness temperature field.
- $A(\Delta\hat{n}, \nu)$ is the primary beam pattern of the telescope.
- $\exp \left[2\pi i \vec{U} \cdot \Delta\hat{n} \right]$ is the phase factor.

In Healpix we discretize the sky and the integral becomes summation. But the integral is restricted only to upper hemisphere.

The discretized version is given as:

$$\mathcal{V}(\alpha_p, \vec{U}, \nu) = Q_\nu \Delta\Omega_{pix} \sum_q T(\hat{n}_q, \nu) A(\Delta\hat{n}_q, \nu) \exp \left[2\pi i \vec{U} \cdot \Delta\hat{n}_q \right]$$

where α_p is the pointing RA. For more details, [see Section \(2\)](#).

Also,

$$\begin{aligned} Q_\nu &= 2k_B/\lambda^2 \\ \Delta\hat{n} &= \hat{n} - \hat{p} \\ \vec{U} &= u \hat{e}_1(\alpha_p) + v \hat{e}_2(\alpha_p) + w \hat{e}_3(\alpha_p) \end{aligned}$$

$\hat{e}_1, \hat{e}_2, \hat{e}_3$ are the **basis** vectors defined in the local tangent plane. $\hat{e}_1$ is along **east**, $\hat{e}_2$ is along **north** and $\hat{e}_3$ is **vertically overhead**, i.e. **zenith pointed**.

The **basis** vectors $\hat{e}_1, \hat{e}_2, \hat{e}_3$ are dependent on pointing **RA** α_p and **declination(DEC)** δ_p . Components of basis vectors in the cartesian system xyz is given by:

$$\begin{aligned}\hat{e}_1 &= [-\sin \alpha_p, \cos \alpha_p, 0] \\ \hat{e}_2 &= [-\cos \alpha_p \sin \delta_p, -\sin \alpha_p \sin \delta_p, \cos \delta_p] \\ \hat{e}_3 &= [\cos \alpha_p \cos \delta_p, \sin \alpha_p \cos \delta_p, \sin \delta_p]\end{aligned}$$

Where $\Delta\Omega_{pix}$ refers to the solid angle subtended by each simulation pixel and $\hat{p}$ refers to the pointing direction of the antenna (here zenith pointed), and hence $\hat{p} = \hat{e}_3$.

Notes

In the simulation of visibilities for 3D sky signal, we have not incorporated the fact that the baselines $\vec{U}$ and antenna beam pattern $A(\Delta\hat{n}, \nu)$ changes as we vary the frequency. We assume only the brightness temperature field $T(\hat{n}, \nu)$ changes with frequency.

1.2 To run:

```
>> import simvis as sv
>> # skysimtype = '2D' for 2D sky signal (multiple realizations)
>> # skysimtype = '3D' for 3D sky signal
>> sv.sim_vis(infits, outfits, skysimtype)
>> # If you want the seed to be, say seed = 5, run:
>> sv.sim_vis(infits, outfits, skysimtype, seed = 5) # seed = None for random.
>> # for 2D sky signal
>> sv.sim_vis(infits, outfits, skysimtype = '2D', seed)
>> # for 3D sky signal
>> sv.sim_vis(infits, outfits, skysimtype = '3D', seed)
```

In generating 2D sky signal (for multiple realizations), for each realization of GRF, a seed is given. By default seed is set to **None**.

If you are doing $N_{\text{realizations}}$, in each realization seed is incremented by 1 w.r.t. previous value. For example, say you want to generate 10 sky maps, i.e. $N_{\text{realizations}} = 10$, and you gave seed = 5(say). Then while doing 1st realization seed will be 5, in next realization seed will increment by 1 i.e. 6 and so on.

For 2D sky signal case:

The entire code works at a single frequency, the centered frequency ν_c , which it reads from the input UVFITS file. It then simulates the visibilities at a RA,DEC which is read from input UVFITS file.

- Mutiple realizations (number of realizations $N_{\text{realizations}}$) are written along the frequency axis of output FITS.
- $N_{\text{realizations}}$ cannot exceed N_{channels} the number of channels in the original UVFITS.
- Channels $N_{\text{realizations}} + 1$ to N_{channels} of are left untouched in the output FITS file.
- The maximum value of angular multipole ℓ_{max} is by default $(3 \times n_{\text{side}}) - 1$.

For 3D sky signal case:

- Only the sky signal is frequency dependent.
- The sky signal and visibilities are being simulated for all the channels present in the input UVFITS file.
- Frequency dependent antenna beam pattern and baselines are not incorporated.

If you want to do some operations with your data in uvinfits file but do not wish to write it in a uvoutfits file you can call the function `sim_vis_hdul`. And it will update the data in the hdulist.

For example run :

```
>> with fits.open(uvinfits) as hdulist:
>>     sv.sim_vis_hdul(hdulist, skysimtype, seed )
>> # put your operations or code function here that will act on hdulist.
>> # if you want to save the data use :
>> hdulist.writeto(uvoutfits, overwrite = True)
```

1.2.1 Angular Power Spectrum Function

If you want to give your own APS function as in the input please make the APS function first and make sure it has only ℓ dependence. Say your APS function is `myaps(l)` and you want to generate the GRF by this APS. To do this run:

```
>> import numpy as np
>> import simvis as sv
>> def myaps(l):
>>     Amp = 50 # in mK^2 unit
>>     beta = -1 # power index
>>     return Amp * np.power(l, beta)
>
>> sv.sim_vis(infits, outfits, skysimtype = '2D' ,
              seed = 5, apsfunc = myaps )
```

The pre-defined APS function `aps` has the form of

$$C_\ell = A \left(\frac{\ell}{\ell_0} \right)^\beta$$

Where A defines the amplitude of the APS is in the unit of mK^2 . β here is the power-law index. C_ℓ has the unit of (mK^2) . By default set to $A = 100 \text{ mK}^2$, $\ell_0 = 1$, $\beta = -2$. (See more in module functions).

1.2.2 Power Spectrum Function

Power spectrum function P^M here is defined as :

$$P^M(k) = A \left(\frac{k}{k_0} \right)^s \quad (1.1)$$

with $A = 1, k_0 = 1, s = -2$ (Default set). A has the unit of $\text{mK}^2 \text{Mpc}^3$. k, k_0 are in Mpc^{-1} unit.

We generate 3D sky signal (temperature field) across all the given frequency channels by :

$$T(\hat{\mathbf{n}}, \nu) = \sum_{\ell, m} a_{\ell m}(\nu) Y_{\ell m}(\hat{\mathbf{n}}) \quad (1.2)$$

where we generate $a_{\ell m}$ coefficients at n^{th} frequency channel by :

$$a_{\ell m}(\nu_n) = \sum_{q=0}^{N_c-1} \left[\sqrt{\frac{P^M(k_{\perp}, k_{\parallel q})}{2N_c \Delta\nu_c r^2 r'}} (\hat{\mathbf{x}}_q + i\hat{\mathbf{y}}_q) \right] \exp \left[\frac{2\pi i n q}{N_c} \right] \quad (1.3)$$

Where -

1. $k_{\perp} = \ell/r$ and $k_{\parallel}$ are the components of comoving wave vector $\vec{k}$ which is perpendicular and parallel (along) to the line of sight respectively.
2. r is the comoving distance at centered frequency ν_c .
3. $r' = \frac{dr}{d\nu}$ evaluated at ν_c .
4. N_c is the number of frequency channels.
5. $\Delta\nu_c$ is the frequency separation between two consecutive frequency channels.
6. ν_n refers to n -th frequency channel. So by convention ν_0, ν_{N_c-1} will be the frequencies of first and last frequency channel.
7. $\hat{\mathbf{x}}_q, \hat{\mathbf{y}}_q \sim \mathcal{N}(0, 1)$ are independent Gaussian random variables of unit variance and by equating $a_{\ell m}(\nu_n)$ s, we generate $T(\hat{\mathbf{n}}, \nu_n)$ by Eq. (1.2).

Notes

As temperature field is a real field, we must remember that this follows a property:

$$a_{\ell, m} = (-1)^m [a_{\ell, -m}]^*$$

where $*$ means complex conjugation. From this property we have if $m = 0$ then we have $a_{\ell, m=0} = [a_{\ell, m=0}]^*$, meaning $a_{\ell, m=0}$ must be real.

So for $m = 0$ case we take the real part of the $a_{\ell, m}$ generated by Eq. (1.3), i.e.

$$a_{\ell}^{m=0}(\nu_n) = \mathcal{Re} \left\{ \sum_{q=0}^{N_c-1} \left[\sqrt{\frac{P^M(k_{\perp}, k_{\parallel q})}{2N_c \Delta\nu_c r^2 r'}} (\hat{\mathbf{x}}_q + i\hat{\mathbf{y}}_q) \right] \exp \left[\frac{2\pi i n q}{N_c} \right] \right\} \quad (1.4)$$

We have the relation between co-moving distance (r) to the red-shift (z) given by:

$$r_\nu = \int_0^z \frac{c \, dz'}{H(z')} \implies \frac{\partial r_\nu}{\partial z} = \frac{c}{H(z)}$$

Where $H(z)$ is the Hubble parameter at red-shift z given by (In Flat Λ CDM model):

$$H(z) = H_0 [\Omega_{m0} (1+z)^3 + \Omega_{\Lambda 0}]^{1/2}$$

We define :

$$\begin{aligned} r'_\nu &= \frac{\partial r_\nu}{\partial \nu} \\ &= \frac{\partial r_\nu}{\partial z} \cdot \frac{\partial z}{\partial \nu} \\ &= -\frac{\nu_0}{\nu^2} \cdot \frac{c}{H(z)} \\ &= -\frac{(1+z)^2}{\nu_0} \cdot \frac{c}{H(z)} \end{aligned}$$

Where we have used $\nu = \nu_0/(1+z)$. Implying $\frac{\partial z}{\partial \nu} = -\frac{\nu_0}{\nu^2} = -\frac{(1+z)^2}{\nu_0}$

$\nu_0 = 1420$ MHz. We take only the absolute value $|r'_\nu| = \frac{(1+z)^2}{\nu_0} \cdot \frac{c}{H(z)}$.

The cosmological parameters are taken from [Planck 18 Astropy](#) , [FlatLambdaCDM](#).

Notes

To generate 3D sky signal, needed parameters **nuc**, **nc**, **dnuc**, r_ν is r' , nc is the no of channels N_c , dnu is channel width $\Delta\nu_c$, nuc is the centered frequency ν_c .

If you want to give your own power spectrum function (make sure it is vectorized)

```
>> import numpy as np
>> import builtins as blt
>> import simvis as sv
>> # make sure all are float.
>> blt.A = 1.0
>> blt.s = -3.0
>> blt.k0 = 1.0
>> @np.vectorize
>> def mypkfunc(k):
>>     if k == 0:
>>         return 0.0
>>     else:
>>         return A*(k/k0)**(s)
>>
>> sv.sim_vis(infits, outfits, skysimtype = '3D',
>>            seed = 5, psfunc = mypkfunc)
```

Notes

- APS function generates 2D sky signal at centered frequency ν_c .
- PS function generates 3D sky signal for all frequency channels available in the **fits file**.

1.2.3 Important point

Before you can simulate, you need to first set some parameters related to simulation.

- `nside` : Healpix parameter that sets the resolution.
- `Nrea` : No of realizations of sky ($N_{\text{realizations}}$).
- `chunk_size` : Number of baselines for which visibilities are being simulated at one step.

`chunk_size` is a parameter which can be varied as per system requirements. If you have higher ram say 32 GB use it as 300. For 16 you can use 200 and for less use 100. This number is for `nside` = 512 and `Nrea` = 100. As in a part of calculations a matrix comes which has the dimensions of $6n_{\text{side}}^2 \times \text{chunk_size}$ with each entry represented by 128 bit complex number. So it takes memory to store it in RAM. This should not be more than RAM capacity. If you have more RAM you cant increase the `chunk_size` indefinitely as matrix operations will take much time to do the computation. I found, for `nside` = 512 and `Nrea` = 100 , `chunk_size` = 300-400 (for 32 GB device) is efficient.

To set them, follow this :

```
>> import simvis as sv
>> import builtins as blt
>>
>> # Set simulation parameters
>> blt.nside = 512          # nside parameter
>> blt.Nrea  = 100         # no of realizations
>> blt.chunk_size = 400    # chunk_size, no of baselines in a single step
```

1.3 Overview

- First we generate the **Gaussian Random Field** $T(\hat{n}, \nu)$ (temperature field in unit of mK) from **Angular Power Spectrum** or **Power Spectrum**.
- Calculate the **primary beam pattern** $A(\Delta\hat{n}, \nu)$ for the telescope.
- Calculate the components of $\hat{n}$ along the basis vectors $\hat{e}_1, \hat{e}_2, \hat{e}_3$. This will help us to determine the dot product $\vec{U} \cdot \Delta\hat{n}$.
- Now we calculate the **phase factor** $\exp \left[2\pi i \vec{U} \cdot \Delta\hat{n} \right]$.
- Multiply the **GRF and PB** first and then multiply by phase factor, sum over the pixels. The whole thing is *done by matrix multiplication*.
- The visibility is simulated.

1.4 Module Functions

`simvis.aps(l)`

This is the input model Angular Power Spectrum(APS) function.

$$C_\ell = A \left(\frac{\ell}{\ell_0} \right)^\beta$$

A defines the amplitude of the APS is in the unit of mK^2 . β here is the power-law index. C_ℓ has the unit of (mK^2) . By default set to $A = 100 \text{ mK}^2$, $\ell_0 = 1$, $\beta = -2$.

Parameters

1 (*float or np.ndarray*) – The angular multipole ℓ value(s).

Returns

The angular power spectrum value for the desired multipole(s) ℓ .

Return type

float or np.ndarray

`simvis.uaps(l)`

This is the Unit Angular Power Spectrum function.

Parameters

1 (*float or np.ndarray*) – The angular multipole ℓ value(s).

Returns

1.0

Return type

float

`simvis.grf(nside, lmax, apsfunc, seed)`

Generates GRF values for every pixel using the input APS(apsfunc).The GRF is monopole subtracted.

Parameters

- `nside` (*int*) – Healpix parameter, usually power of 2.
- `lmax` (*int*) – Maximum value of angular multipole(ℓ)
- `apsfunc` (*function*) – The input APS function.
- `seed` (*int or float*) – A seed value, which can be provided to get different realizations for same APS. If you pass the seed value, then the output GRF is a fixed one.

Returns

Contains the GRF value for each pixels. Shape (12 nside^2) . Returned values are in mK unit.

Return type

np.ndarray

`simvis.ps(k)`

This is the input model Power Spectrum function, which has the form $A \left(\frac{k}{k_0} \right)^s$, with $A = 1, k_0 = 1, s = -2$ (Default set). A has the unit of $\text{mK}^2 \text{Mpc}^3$. k, k_0 are in Mpc^{-1} unit.

Parameters

k (*float or np.ndarray*) – $k = |\vec{k}|$, the comoving wave number.

Returns

The value of Power Spectrum $P(k)$ at comoving wave number k .

Return type

float or np.ndarray

Notes

Returns 0.0 if $k = 0$.

`simvis.sky3dgrf(nside, seed, psfunc=<function ps>)`

This function simulates the 3D sky signal for a given `nside`, Power Spectrum function, seed value across all the frequency channels taken from input UVFITS file.

Parameters

- `nside` (*int*) – Healpix parameter (usually power of 2)
- `seed` (*Positive int or None*) – seed value for generating the $a_{\ell m}$'s. Different seed value will result in different $a_{\ell m}$'s, but for a particular seed value that will be fixed.
- `psfunc` (*function*) – The input Power Spectrum function. By default it is `ps` which has the form $A \left(\frac{k}{k_0} \right)^s$, with $A = 1, k_0 = 1, s = -2$. You can define your own power spectrum function make sure that it has only k dependence.

Returns

2D array has the shape $N_c \times \text{npix}$. Each row data corresponds to GRF at a particular frequency. `npix` is the number of pixels. To get the value of `npix` see notes section below.(usually it is 12nside^2).

Return type

Array type

Notes

By default $\ell_{max} = 3 * nside - 1$. To extract gaussian random field value $T(\hat{n}, \nu)$ for a given channel no or no's (indexing starts from 0).

```
>> import healpy as hp
>> nside, seed = 16, None
>> npix = hp.nside2npix(nside)
>> print(npix)
>> channel = 10
>> # for channels from 1 to 10
>> channel = [ i for i in range(10)]
>> grf = sky3dgrf(nside, seed) # generate the grf
>> grf[channel] # extract the values
```

GRF across all channels has no monopole, as $a_{0,0}$ is set zero.

If you want to give your own power spectrum function (make sure it is vectorized)

```
>> import numpy as np
>> import builtins as blt
>> import simvis as sv
>> # make sure all are float.
>> blt.A = 1.0
>> blt.s = -3.0
>> blt.k0 = 1.0
>> @np.vectorize
>> def mypkfunc(k):
>>     if k == 0:
>>         return 0.0
>>     else:
>>         return A*(k/k0)**(s)
>>
>> grf = sv.sky3dgrf(nside, seed, psfunc = mypkfunc) # generate the grf
>>
```

`simvis.allskysim(inside, seed, skysimtype, apsfunc=<function aps>, psfunc=<function ps>)`

This function creates all sky maps accordingly.

Parameters

- `inside` (*int*) – Healpix parameter. Usually power of 2.
- `seed` (*Positive int or None*) – If its 2D map then in each realization the seed gets increased by 1.
- `skysimtype` (*str*) – Either 2D or 3D. If given 2D then creates multiple realizations. If given 3D then makes a 3D sky map.
- `apsfunc` (*function*) – The input Angular Power Spectrum function. By default it is `aps` which has the form $C_\ell = A \left(\frac{\ell}{\ell_0}\right)^\beta$ with $A = 100 \text{ mK}^2$, $\ell_0 = 1$, $\beta = -2$.
- `psfunc` (*function*) – The input Power Spectrum function. By default it is `ps` which has the form $A \left(\frac{k}{k_0}\right)^s$, with $A = 1$, $k_0 = 1$, $s = -2$.

Returns

Surface brightness temperature field map (multiple realizations or across all frequencies) in units of mK.

Return type

`np.ndarray`

`simvis.beammwa(nu, ne1, ne2)`

We model primary beam function of **MWA telescope** as a product of two sinc square function given as :

$$\mathcal{A}(\hat{\mathbf{n}}, \nu) = \text{sinc}^2 \left[\frac{\pi b \nu}{c} \Delta \hat{\mathbf{n}} \cdot \hat{\mathbf{e}}_1(\alpha_p) \right] \text{sinc}^2 \left[\frac{\pi b \nu}{c} \Delta \hat{\mathbf{n}} \cdot \hat{\mathbf{e}}_2(\alpha_p) \right]$$

Where :

$$\Delta \hat{\mathbf{n}} = \hat{\mathbf{n}} - \hat{\mathbf{p}}$$

$\hat{\mathbf{p}}$ is the pointing direction of the telescope. We assume that telescope is zenith pointed ($\hat{\mathbf{p}} = \hat{\mathbf{e}}_3$). $\hat{\mathbf{n}}$ refers to different directions on the sky (i.e. celestial sphere).

Parameters

- `nu` (*float*) – Observing frequency ν in MHz unit.
- `ne1` (*float or array*) – The dot product given by $\hat{\mathbf{n}} \cdot \hat{\mathbf{e}}_1$.
- `ne2` (*float or array*) – The dot product given by $\hat{\mathbf{n}} \cdot \hat{\mathbf{e}}_2$.

Returns

The primary beam pattern value for square aperture telescope (MWA).

Return type

`float or np.ndarray`

`simvis.pbgen(inside, ra_ptg, dec_ptg, nu, pbfunction)`

This generates the primary beam for MWA telescope (zenith pointed), given the RA,DEC and frequency.

Parameters

- `inside` (*int*) – Healpix parameter. Usually power of 2.
- `ra_ptg` (*int or float*) – RA of the pointing direction. (In degrees) (zenith pointed).
- `dec_ptg` (*int or float*) – DEC of the pointing direction. (In degrees) (zenith pointed).
- `nu` (*float*) – Observing frequency ν in MHz unit.
- `pbfunction` (*function*) – The primary beam (PB) pattern of MWA which is product of two sinc^2 functions.

Returns

PB – The value of primary beam of the MWA telescope (beam pattern value at each pixel which quantifies the response of telescope along different directions). Shape $(12 \text{ inside}^2,)$.

Return type

`np.ndarray`

`simvis.hat_n(inside, ipix)`

This calculates the xyz components of $\hat{n}$ for all the given pixels contained in the array `ipix`.

Parameters

- `inside` (*int*) – Healpix parameter. Usually power of 2.
- `ipix` (*np.ndarray*) – Pixel array for which components of $\hat{n}$ have to be calculated.

Returns

hatn – Containing the xyz comps of $\hat{n}$. $N_{pix} \times 3$.

Return type

`np.ndarray`

`simvis.dot_cal_superfast(inside, ra_ptg, dec_ptg, ipix)`

This calculates components of $\Delta\hat{n}$ for all the given pixels contained in the array `ipix` along the basis vectors $\hat{e}_1, \hat{e}_2, \hat{e}_3$ given by the RA,DEC.

Parameters

- `inside` (*int*) – Healpix parameter. Usually power of 2.
- `ra_ptg` (*int or float*) – RA of the pointing direction. (In degrees)
- `dec_ptg` (*int or float*) – DEC of the pointing direction. (In degrees)
- `ipix` (*np.ndarray*) – Pixel array for which dot products have to be calculated.

Returns

dot_products – Contains the xyz comps for those pixels. Shape $N_{pix} \times 3$.

Return type

`np.ndarray`

`simvis.calculate_phase(dot_product, bl)`

Given the baselines it calculates the phase factor for all the pixels given in `dot_product` array.

Parameters

- `dot_product` (*np.ndarray*) – Contains the components for all the given those pixels along the basis vectors $\hat{e}_1, \hat{e}_2, \hat{e}_3$. Shape is $N_{pix} \times 3$.
- `bl` (*np.ndarray*) – The array contains the components of the baselines. (defined by basis vectors $\hat{e}_1, \hat{e}_2, \hat{e}_3$). Shape $N_{Baselines} \times 3$.

Returns

The phase factor. Shape $N_{pix} \times N_{Baselines}$.

Return type

`np.ndarray`

`simvis.visgen_mwa_multi(nside, grfmap, ra_ptg, dec_ptg, bl, nu)`

This calculates the visibilities for the baselines passed by `bl` array, given the RA, DEC, frequency, `grfmap`.

Parameters

- `nside` (*int*) – Healpix parameter. Usually power of 2.
- `grfmap` (*np.ndarray*) – Contains the sky brightness temperature simulated from APS or PS for all the pixels. (Contains multiple realizations for 2D map or sky signal across all frequencies for 3D map). Shape $N_{Realizations} \times 12 \text{ nside}^2$ for 2D, $N_c \times 12 \text{ nside}^2$ for 3D.
- `ra_ptg` (*int or float*) – RA of the pointing direction. (In degrees)
- `dec_ptg` (*int or float*) – DEC of the pointing direction. (In degrees)
- `bl` (*np.ndarray*) – The array contains the components of the baselines defined by basis vectors $\hat{e}_1, \hat{e}_2, \hat{e}_3$. Shape $N_{Baselines} \times 3$.
- `nu` (*float*) – Observing frequency ν in MHz unit.

Returns

vis – Contains the simulated visibility for all the baselines given by `bl`. (Along axis = 0 is the different realizations or frequencies, axis = 1 is the baselines). Shape $N_{realizations} \times N_{Baselines}$ for 2D, $N_c \times N_{Baselines}$ for 3D.

Return type

`np.ndarray`

`simvis.sim_vis_hdul(hdulist, skysimtype, seed=None, apsfunc=<function aps>, psfunc=<function ps>)`

This calculates the visibilities for the baselines passed by `bl`, given the RA, DEC, frequency. It extracts the necessary information from the `hdulist`, like the baseline distribution, centered frequency, frequency resolution, no of frequency channels, RA, DEC.

Parameters

- `hdulist (object)` – A fits object
- `skysimtype (str)` – Either 2D or 3D. If given 2D then creates multiple realizations. If given 3D then makes a 3D sky map.
- `seed (int or float)` – A seed value for generating GRF. (By default set to None)
- `apsfunc (function)` – The input Angular Power Spectrum function. By default it is `aps` which has the form $C_\ell = A \left(\frac{\ell}{\ell_0}\right)^\beta$ with $A = 100 \text{ mK}^2$, $\ell_0 = 1$, $\beta = -2$.
- `psfunc (function)` – The input Power Spectrum function. By default it is `ps` which has the form $A \left(\frac{k}{k_0}\right)^s$, with $A = 1$, $k_0 = 1$, $s = -2$.

Return type

Simulates the visibilities for the input Power Spectrum or Angular Power Spectrum for the RA, DEC, frequency and baseline distribution given in the `hdulist` object. (And puts the multiple realizations along the frequency axis if simulated sky signal is 2D).

```
simvis.sim_vis(uvinfits, uvoutfits, skysimtype, seed=None, apsfunc=<function aps>, psfunc=<function
ps>)
```

This calculates the visibilities for the baselines passed by `bl_file`, given the RA, DEC, frequency. It extracts the necessary information from the `uvinfits` file, like the baseline distribution, centered frequency, frequency resolution, no of frequency channels, RA, DEC and writes the simulated visibilities in a new fits file named `uvoutfits` file.

Parameters

- `uvinfits (fits file)` – Input fits File.
- `uvoutfits (fits file)` – Output fits File.
- `skysimtype (str)` – Either 2D or 3D. If given 2D then creates multiple realizations. If given 3D then makes a 3D sky map.
- `seed (int or float)` – A seed value for generating GRF.
- `apsfunc (function)` – The input Angular Power Spectrum function. By default it is `aps` which has the form $C_\ell = A \left(\frac{\ell}{\ell_0} \right)^\beta$ with $A = 100 \text{ mK}^2$, $\ell_0 = 1$, $\beta = -2$.
- `psfunc (function)` – The input Power Spectrum function. By default it is `ps` which has the form $A \left(\frac{k}{k_0} \right)^s$, with $A = 1$, $k_0 = 1$, $s = -2$.

Return type

Simulates the visibilities for the input Power Spectrum or Angular Power Spectrum for the RA, DEC, frequency and baseline distribution given in the `uvinfits` file. (And puts the multiple realizations along the frequency axis if simulated sky signal is 2D). Writes the data in `uvoutfits` file.